A One numerical target; two proposal mechanisms

Greedy verification with symbolic tokens; scheduling is illustrative, not a measured trace

Committed prefix

`x(t-3)` `x(t-2)` `x(t-1)` `x(t)`

Reusable target KV cache for committed context

Native MTP

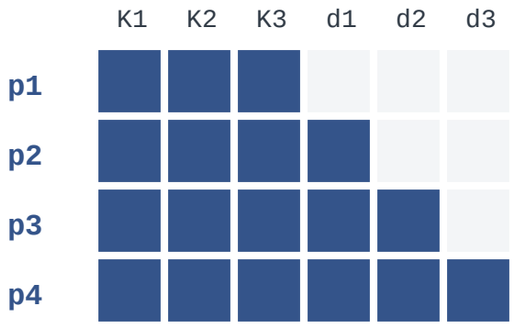
Use preserved mtp.* tensors;
propose d1, d2, d3 in a draft loop.

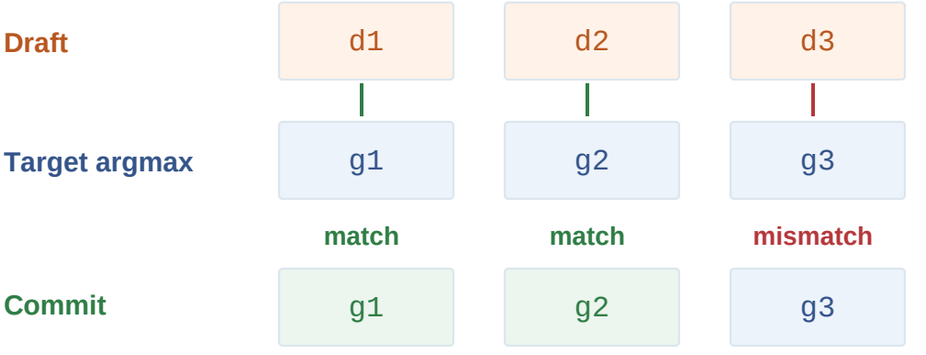
N-gram lookup

Match a context suffix;
reuse following tokens as proposals.

B Parallel target verification

Target distributions follow the same prefix and the proposed causal token history





Visibility mask, not attention scores

K1-K3 symbolize the cached prefix.
Each p(i) predicts a next token.
New query positions are target-batched.
Backend cache scheduling may differ.

Accept the longest agreeing prefix; replace at the first mismatch.
Discard the rejected and dependent draft suffix.
All accepted? The target can supply a bonus token.
Only verified, parsed output can dispatch a tool.

C What can become faster, and what stays fixed

Separate image-prefill, token decoding and real tool execution

Logical operations: widths do not encode latency or a claimed speedup.

Recorded per episode / request

```
wall_seconds tool_seconds
prefill_seconds decode_seconds
generated_tokens accepted / drafted
GPU peaks raw outputs + tool calls
```

D Evidence gates and execution envelope

Baseline and speculation use the same merged checkpoint; never compare across numeric targets

BEFORE: compare mtp.* keys, shapes, dtypes and bytes with the original native base.
AFTER: compare decoded turns, tool calls, final answers and pooled dev exact F1.
FAILURE: retain divergences; suppress equivalence-qualified speedup claims.

Requested: dual T4 = FP16 / TP2; RTX 4090 and 48-GB RTX Pro = BF16 / TP1.
Fit/support need a real runtime probe. This runner does not supply TPU speculation.